LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 9



IMMANUEL RADITYA DEO PRATAMA PAMBUDHI

103132430001

S1SD-05-A

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

Dasar Teori

Modul 9 membahas empat konsep inti:

1. Static Variable & Method: Variabel dan method milik kelas, diakses via NamaKelas tanpa perlu new. Static variable dibagi semua instance dan static method tidak bisa akses member non-static.

2. Collection Framework: Arsitektur standar untuk mengelola grup objek dengan Interface utama: List, Set (unik), Queue (antrian). Implementasi umumnya: ArrayList (akses indeks cepat), HashSet (unik), LinkedList (sisip/hapus cepat). Generics (<T>) memastikan type safety pada compile-time.

3. Iterasi Koleksi dengan lima cara: for-indeks (akses indeks), for-each (read-only bersih), Iterator (aman hapus saat loop), lambda (ringkas fungsional), method reference.

4. Sorting dengan Comparable (urutan natural/bawaan via compareTo, satu strategi), Comparator (urutan kustom via compare, banyak strategi tanpa ubah kelas asli). Aturan compareTo: String/Date pakai compareTo, angka pakai Integer.compare/double.compare.

5. Filtering & Stream API: Filtering konvensional pakai loop + if; Stream API pakai filter(), collect(), findFirst() lebih deklaratif, ringkas, dan terbaca.

Guided

Guided 1: Static Variabel dan Method (Share.java, Driver.java)

Share.java:

```java
package guided.gd1;

public class Share {
    private int privateInt;
    private static int staticInt;

    public Share(int pr, int si){
        privateInt = pr;
        staticInt = si;
    }
```

```
    public String toString(){
        return privateInt + " " + staticInt;
    }
}
```

Driver.java:

```
package guided.gd1;

public class Driver {

    public static void main(String[] args) {
        Share s1 = new Share(4, 4);
        System.out.println(s1.toString());

        Share s2 = new Share(8, 2);
        System.out.println(s1.toString());
        System.out.println(s2.toString());

        Share s3 = new Share(6, 22);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        System.out.println(s3.toString());
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>java guided.gd1.Driver
4 4
4 2
8 2
4 22
8 22
6 22

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>_
```

Guided 2: Static Counter (CounterMachine.java, Driver.java)

CounterMachine.java:

```java
package guided.gd2;

public class CounterMachine {
    static int counter = 0;
    public static void count(){
        counter++;
    }
}
```

Driver.java:

```java
package guided.gd2;

public class Driver {

    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            if (i % 2 == 0) {
                CounterMachine.count();
            }
        }
        System.out.println("Counter: " + CounterMachine.counter);
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>java guided.gd2.Driver
Counter: 5

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>_
```

Guided 3: Filtering, Searching, Iteration, dan Sorting

3.1 Filtering & Searching (Mahasiswa.java, MainFilter.java)

Mahasiswa.java:

```
package guided.gd3.filtering;

public class Mahasiswa {
    private String nim;
    private String nama;
    private double ipk;

    public Mahasiswa(String nim, String nama, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.ipk = ipk;
    }

    public String getNim() { return nim; }
    public String getNama() { return nama; }
    public double getIpk() { return ipk; }

    @Override
    public String toString() {
        return String.format("Mhs[NIM=%s, Nama=%-6s, IPK=%.2f]", nim, nama,
ipk);
    }
}
```

MainFilter.java:

```
package guided.gd3.filtering;

import java.util.ArrayList;
```

```
import java.util.List;
import java.util.stream.Collectors;

public class MainFilter {
    public static void main(String[] args) {
        List<Mahasiswa> list = new ArrayList<>();
        list.add(new Mahasiswa("001", "Bobby", 3.25));
        list.add(new Mahasiswa("002", "Erick", 2.80));
        list.add(new Mahasiswa("003", "Rey", 3.75));
        list.add(new Mahasiswa("004", "Anna", 3.90));

        System.out.println("=== 1. FILTERING (IPK >= 3.50) ===");

        System.out.println("Cara Konvensional:");
        List<Mahasiswa> mhsCumlaudeKonven = new ArrayList<>();
        for (Mahasiswa m : list) {
            if (m.getIpk() >= 3.50) {
                mhsCumlaudeKonven.add(m);
            }
        }
        for (Mahasiswa m : mhsCumlaudeKonven) {
            System.out.println(m);
        }

        System.out.println("
Cara Stream API:");
        List<Mahasiswa> mhsCumlaudeStream = list.stream()
                .filter(m -> m.getIpk() >= 3.50)
                .collect(Collectors.toList());
        mhsCumlaudeStream.forEach(System.out::println);

        System.out.println("
=== 2. SEARCHING / PENCARIAN TUNGGAL ===");

        System.out.println("Cara Konvensional (Cari 'Erick'):");
        Mahasiswa targetKonven = null;
        for (Mahasiswa m : list) {
```

```
            if (m.getNama().equalsIgnoreCase("Erick")) {
                targetKonven = m;
                break;
            }
        }
        System.out.println(targetKonven != null ? targetKonven : "Data tidak
ditemukan");

        System.out.println("
Cara Stream API (Cari 'Erick'):");
        Mahasiswa targetStream = list.stream()
                .filter(m -> m.getNama().equalsIgnoreCase("Erick"))
                .findFirst()
                .orElse(null);
        System.out.println(targetStream != null ? targetStream : "Data tidak
ditemukan");
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>java guided.gd3.filtering.MainFilter
=== 1. FILTERING (IPK >= 3.50) ===
Cara Konvensional:
Mhs[NIM=003, Nama=Rey   , IPK=3,75]
Mhs[NIM=004, Nama=Anna  , IPK=3,90]

Cara Stream API:
Mhs[NIM=003, Nama=Rey   , IPK=3,75]
Mhs[NIM=004, Nama=Anna  , IPK=3,90]

=== 2. SEARCHING / PENCARIAN TUNGGAL ===
Cara Konvensional (Cari Erick):
Mhs[NIM=002, Nama=Erick , IPK=2,80]

Cara Stream API (Cari Erick):
Mhs[NIM=002, Nama=Erick , IPK=2,80]

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>_
```

3.2 Iterasi Koleksi (Main.java)

Main.java:

```
package guided.gd3.iteration;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> listBuah = new ArrayList<>();

        listBuah.add("Apel");
        listBuah.add("Mangga");
        listBuah.add("Jeruk");
        listBuah.add("Pisang");

        System.out.println("Total Elemen: " + listBuah.size());
        System.out.println("Apakah ada 'Mangga'? " +
listBuah.contains("Mangga"));

        listBuah.remove("Pisang");

        System.out.println("
--- 1. Normal For-Loop (Indeks) ---");
            for (int i = 0; i < listBuah.size(); i++) {
                System.out.println("Index " + i + ": " + listBuah.get(i));
        }

        System.out.println("
--- 2. Enhanced For-Loop (For-Each) ---");
        for (String buah : listBuah) {
            System.out.println(buah);
        }

        System.out.println("
```

```
--- 3. Menggunakan Iterator ---");
        Iterator<String> itr = listBuah.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }

        System.out.println("
--- 4. Lambda Expression ---");
        listBuah.forEach(buah -> System.out.println(buah));

        System.out.println("
--- 5. Method Reference ---");
        listBuah.forEach(System.out::println);
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>java guided.gd3.iteration.Main
Total Elemen: 4
Apakah ada Mangga? true

--- 1. Normal For-Loop (Indeks) ---
Index 0: Apel
Index 1: Mangga
Index 2: Jeruk

--- 2. Enhanced For-Loop (For-Each) ---
Apel
Mangga
Jeruk

--- 3. Menggunakan Iterator ---
Apel
Mangga
Jeruk

--- 4. Lambda Expression ---
Apel
Mangga
Jeruk

--- 5. Method Reference ---
Apel
Mangga
Jeruk

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>_
```

3.3 Sorting (Employee.java, SalaryComparator.java, MainSort.java)

Employee.java:

```java
package guided.gd3.sorting;

public class Employee implements Comparable<Employee> {
    private String name;
    private int salary;

    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }

    public String getName() { return name; }
    public int getSalary() { return salary; }

    @Override
    public int compareTo(Employee other) {
        return this.name.compareTo(other.name);
    }

    @Override
    public String toString() {
        return String.format("Employee[name=%-6s, salary=%d]", name, salary);
    }
}
```

SalaryComparator.java:

```java
package guided.gd3.sorting;

import java.util.Comparator;

public class SalaryComparator implements java.util.Comparator<Employee> {
    @Override
```

```
    public int compare(Employee e1, Employee e2) {
        return Integer.compare(e1.getSalary(), e2.getSalary());
    }
}
```

MainSort.java:

```
package guided.gd3.sorting;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class MainSort {
    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList<>();
        listEmp.add(new Employee("Bobby", 3000));
        listEmp.add(new Employee("Erick", 1600));
        listEmp.add(new Employee("Rey", 2500));
        listEmp.add(new Employee("Anna", 3500));

        System.out.println("--- Data Awal ---");
        listEmp.forEach(System.out::println);

        Collections.sort(listEmp);
        System.out.println("
--- Sorted by Name (Comparable / Natural Order) ---");
        listEmp.forEach(System.out::println);

        Collections.sort(listEmp, new SalaryComparator());
        System.out.println("
--- Sorted by Salary (Comparator / Custom Order) ---");
        listEmp.forEach(System.out::println);
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>java guided.gd3.sorting.MainSort
--- Data Awal ---
Employee[name=Bobby , salary=3000]
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]
Employee[name=Anna  , salary=3500]

--- Sorted by Name (Comparable / Natural Order) ---
Employee[name=Anna  , salary=3500]
Employee[name=Bobby , salary=3000]
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]

--- Sorted by Salary (Comparator / Custom Order) ---
Employee[name=Erick , salary=1600]
Employee[name=Rey   , salary=2500]
Employee[name=Bobby , salary=3000]
Employee[name=Anna  , salary=3500]

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>_
```

Unguided

Soal: Manajemen Hasil Eksperimen Model Machine Learning

Program mengelola catatan hasil eksperimen model ML dengan fitur static counter, sorting (Comparable & Comparator), filtering, dan searching menggunakan Stream API.

Jawaban

ModelEksperimen.java:

```java
package unguided;

import java.util.Locale;

public class ModelEksperimen implements Comparable<ModelEksperimen> {
    private String namaModel;
    private double akurasi;
    private int waktuTraining;
    private static int jumlahEksperimen = 0;

    public ModelEksperimen(String namaModel, double akurasi, int waktuTraining)
{
```

```
        this.namaModel = namaModel;
        this.akurasi = akurasi;
        this.waktuTraining = waktuTraining;
        jumlahEksperimen++;
    }

    public String getNamaModel() { return namaModel; }
    public double getAkurasi() { return akurasi; }
    public int getWaktuTraining() { return waktuTraining; }

    public static int getJumlahEksperimen() {
        return jumlahEksperimen;
    }

    @Override
    public int compareTo(ModelEksperimen other) {
        return Double.compare(other.akurasi, this.akurasi);
    }

    @Override
    public String toString() {
        return String.format(Locale.US, "Model=%s, akurasi=%.2f, waktu=%d
detik", namaModel, akurasi, waktuTraining);
    }
}
```

WaktuTrainingComparator.java:

```
package unguided;

import java.util.Comparator;

public class WaktuTrainingComparator implements Comparator<ModelEksperimen> {
    @Override
    public int compare(ModelEksperimen m1, ModelEksperimen m2) {
        return Integer.compare(m1.getWaktuTraining(), m2.getWaktuTraining());
```

```
    }
}
```

Main.java:

```
package unguided;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.stream.Collectors;

public class Main {
    public static void main(String[] args) {
        List<ModelEksperimen> daftarModel = new ArrayList<>();
        daftarModel.add(new ModelEksperimen("Logistic Regression", 0.82, 12));
        daftarModel.add(new ModelEksperimen("Random Forest", 0.91, 45));
        daftarModel.add(new ModelEksperimen("SVM", 0.87, 60));
        daftarModel.add(new ModelEksperimen("KNN", 0.79, 5));
        daftarModel.add(new ModelEksperimen("Decision Tree", 0.84, 8));

        System.out.println("Jumlah eksperimen: " +
ModelEksperimen.getJumlahEksperimen());
        System.out.println();

        System.out.println("Sorted by akurasi");
        Collections.sort(daftarModel);
        daftarModel.forEach(System.out::println);
        System.out.println();

        System.out.println("Sorted by waktu training");
        Collections.sort(daftarModel, new WaktuTrainingComparator());
        daftarModel.forEach(System.out::println);
        System.out.println();

        System.out.println("Model dengan akurasi minimal 0.85");
```

```
        List<ModelEksperimen> filterAkurasi = daftarModel.stream()
                .filter(m -> m.getAkurasi() >= 0.85)
                .collect(Collectors.toList());
        filterAkurasi.forEach(System.out::println);
        System.out.println();

        System.out.println("Pencarian model KNN");
        ModelEksperimen knn = daftarModel.stream()
                .filter(m -> m.getNamaModel().equals("KNN"))
                .findFirst()
                .orElse(null);
        System.out.println(knn);
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>java unguided.Main
Jumlah eksperimen: 5

Sorted by akurasi
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik
Model=Decision Tree, akurasi=0.84, waktu=8 detik
Model=Logistic Regression, akurasi=0.82, waktu=12 detik
Model=KNN, akurasi=0.79, waktu=5 detik

Sorted by waktu training
Model=KNN, akurasi=0.79, waktu=5 detik
Model=Decision Tree, akurasi=0.84, waktu=8 detik
Model=Logistic Regression, akurasi=0.82, waktu=12 detik
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik

Model dengan akurasi minimal 0.85
Model=Random Forest, akurasi=0.91, waktu=45 detik
Model=SVM, akurasi=0.87, waktu=60 detik

Pencarian model KNN
Model=KNN, akurasi=0.79, waktu=5 detik

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-09-static-class-dan-collection\project\src\main\java>_
```

Penjelasan

Pada tugas unguided ini, saya mengembangkan program manajemen eksperimen *machine learning* untuk mengimplementasikan konsep Modul 9 secara menyeluruh:

- Static Variable & Method: Variabel jumlahEksperimen berfungsi sebagai pencatat global di kelas ModelEksperimen yang otomatis bertambah setiap ada objek baru. Nilainya bisa langsung diakses tanpa instansiasi lewat ModelEksperimen.getJumlahEksperimen().
- Collection & Generics: Penerapan ArrayList<ModelEksperimen> memastikan *type safety* agar hanya objek yang valid yang masuk ke dalam list, sekaligus mencegah *casting error* saat *runtime*.
- Comparable (Natural Order): Kelas ModelEksperimen mengimplementasikan Comparable untuk menetapkan urutan bawaan berdasarkan akurasi tertinggi (*descending*) via Double.compare(). Saat Collections.sort() dipanggil tanpa parameter tambahan, data otomatis terurut dari model terbaik.
- Comparator (Custom Order): Pengurutan alternatif dipisahkan ke dalam kelas WaktuTrainingComparator untuk menyortir waktu *training* tercepat (*ascending*) via Integer.compare(). Pemisahan ini menjaga kode tetap modular tanpa perlu mengubah kelas domain utama.
- Stream API (Filter & Search): Proses seleksi model berakurasi ≥ 0.85 dilakukan secara deklaratif menggunakan filter().collect(). Sedangkan pencarian model spesifik (seperti "KNN") ditangani ringkas lewat findFirst().orElseNull(), menggantikan perulangan manual konvensional agar kode lebih bersih dan mudah dibaca.

Kesimpulan

Berdasarkan praktikum modul 9, kesimpulan yang aku peroleh:

1. Static Variable/Method cocok untuk data dan perilaku global per-kelas (counter, utility, konstanta). Static variable dibagi semua instance; static method diakses via NamaKelas tanpa new.
2. Collection (ArrayList) dengan Generics menyediakan struktur data dinamis, type-safe, dan kaya method utilitas (add, size, contains, remove, forEach).
3. Iterasi punya lima pilihan: for-indeks (akses indeks), for-each (read-only bersih), Iterator (aman hapus saat loop), lambda (ringkas fungsional), method reference (paling ringkas).
4. Sorting: Comparable untuk urutan default/bawaan kelas (satu strategi via compareTo). Comparator untuk strategi tambahan/fleksibel (compare) tanpa ubah kelas asli.
5. Stream API dengan lambda (filter, collect, findFirst) lebih deklaratif, ringkas, dan terbaca dibanding loop imperatif konvensional untuk filtering dan searching.